내 실기 요약 - 시험 직전 핵심

다운로드할 때 내 목록에 남아 있는 카드만 개인 순서대로 담았습니다. 왼쪽 열부터 읽고 오른쪽 열로 이어 확인하세요. 계산 예제는 남아 있는 관련 카드에 맞춰 마지막에 모았습니다.

생성 패턴

Factory Method (팩토리 메서드) | 공통 생성 인터페이스를 두고 구체 객체의 종류는 하위 클래스가 결정합니다.

Abstract Factory (추상 팩토리) | 서로 호환되는 여러 객체를 한 제품군으로 묶어 일관된 조합으로 생성합니다.

Builder (빌더) | 복잡한 객체의 생성 과정을 단계로 나누고 필요한 구성 요소를 조립합니다.

Prototype (프로토타입) | 설정이 끝난 기존 객체를 복제하여 같은 구조의 새 객체를 생성합니다.

Singleton (싱글톤) | 인스턴스를 하나만 유지하여 프로그램 전체에서 같은 객체를 공유합니다.

구별: Factory Method는 한 객체의 생성 결정을 하위 클래스에 맡기고, Abstract Factory는 관련된 여러 제품을 한 제품군으로 만듭니다.

구조 패턴

Adapter (어댑터) | 기존 인터페이스를 호출자가 기대한 형태로 바꾸어 두 객체를 연결합니다.

Bridge (브리지) | 추상화와 구현을 별도 계층으로 나누어 두 변화 축을 독립적으로 확장합니다.

Composite (컴포지트) | 개별 객체와 객체 묶음을 같은 인터페이스로 다루어 트리 구조를 처리합니다.

Decorator (데코레이터) | 기존 객체를 같은 인터페이스로 감싸 실행 중 필요한 기능을 추가합니다.

Facade (퍼사드) | 여러 하위 시스템의 복잡한 호출을 하나의 단순한 인터페이스로 제공합니다.

Flyweight (플라이웨이트) | 많은 객체의 공통 상태를 공유하여 유사 객체들의 메모리 사용을 줄입니다.

Proxy (프록시) | 실제 객체와 같은 인터페이스의 대리 객체가 접근 권한과 생성 시점을 관리합니다.

구별: Adapter는 인터페이스를 바꾸고, Decorator는 기능을 더하며, Proxy는 접근을 대신 관리합니다.

행위 패턴

Chain of Responsibility (책임 연쇄) | 여러 처리기를 연결하고 해결하지 못한 요청은 다음 처리기로 넘깁니다.

Command (커맨드) | 실행할 요청을 객체에 담아 저장하거나 취소할 수 있는 형태로 만듭니다.

Interpreter (인터프리터) | 간단한 문법을 클래스로 표현하고 문장을 해석하여 결과를 계산합니다.

Iterator (이터레이터) | 컬렉션의 내부 구조를 노출하지 않고 요소를 순차적으로 탐색합니다.

Mediator (중재자) | 객체들이 서로 직접 통신하지 않고 중재자 객체를 통해 협력하게 합니다.

Memento (메멘토) | 객체의 상태를 저장해 두었다가 나중에 이전 상태로 복원합니다.

Observer (옵저버) | 한 객체의 상태가 바뀌면 등록된 여러 구독 객체에 알려 함께 갱신합니다.

State (상태) | 현재 상태를 담당하는 객체에 맡겨 같은 요청도 상태별로 다르게 처리합니다.

Strategy (전략) | 같은 목적의 알고리즘을 독립 객체로 만들어 상황에 맞는 구현으로 교체합니다.

Template Method (템플릿 메서드) | 상위 클래스가 실행 순서를 정하고 일부 단계의 구현은 하위 클래스에 맡깁니다.

Visitor (방문자) | 객체 구조의 원소 클래스를 바꾸지 않고 새로운 연산을 추가합니다.

구별: Observer는 다른 객체들에 변화를 알리고, State는 한 객체의 현재 상태에 따라 자기 행동을 바꿉니다.

요구사항 종류

기능 요구사항 (Functional) | 로그인·조회처럼 시스템이 수행할 기능과 필요한 입력·출력을 정의합니다.

비기능 요구사항 (Non-functional) | 응답 시간·보안 수준·자원 한도 등 시스템의 품질과 운영 제약을 정의합니다.

사용자 요구사항 (User) | 사용자의 업무 목표와 서비스에 기대하는 기능을 이해하기 쉬운 말로 씁니다.

시스템 요구사항 (System) | 사용자 요구를 구현하고 검증할 수 있도록 구체적인 동작과 조건으로 명세합니다.

구조 다이어그램

클래스 다이어그램 (Class) | 클래스의 속성·연산과 클래스 사이의 관계를 표현합니다.

객체 다이어그램 (Object) | 특정 시점에 존재하는 객체와 속성 값, 객체 간 링크를 표현합니다.

컴포넌트 다이어그램 (Component) | 컴포넌트와 제공·요구 인터페이스 사이의 의존 관계를 표현합니다.

배치 다이어그램 (Deployment) | 실행 컴포넌트가 물리적인 노드에 어떻게 배치되는지 표현합니다.

패키지 다이어그램 (Package) | 모델 요소를 패키지로 묶고 패키지 간 의존 관계를 표현합니다.

행위·상호작용 다이어그램

유스케이스 다이어그램 (Use Case) | 액터와 시스템 기능 사이의 상호작용을 사용자 목표 중심으로 표현합니다.

시퀀스 다이어그램 (Sequence) | 시간 순서에 따른 객체 간 메시지 호출과 응답을 표현합니다.

커뮤니케이션 다이어그램 (Communication) | 객체 간 메시지와 연결 관계를 번호로 표현합니다.

활동 다이어그램 (Activity) | 업무나 연산의 흐름, 조건 분기, 병렬 처리를 표현합니다.

상태 다이어그램 (State) | 객체가 사건에 따라 상태를 바꾸는 생명주기를 표현합니다.

UML 관계

연관 (Association) | 두 객체가 서로 연결되어 정보를 주고받는 관계이며 상속과는 구별됩니다.

일반화 (Generalization) | 하위 요소가 상위의 특성을 물려받으며 상위 요소의 한 종류에 해당합니다.

의존 (Dependency) | 다른 요소를 일시적으로 사용하므로 사용 대상의 변경에 영향을 받습니다.

집합 (Aggregation) | 전체가 부분을 포함하지만 부분은 전체 없이도 독립적으로 존재할 수 있습니다.

합성 (Composition) | 전체가 부분을 강하게 소유하여 전체가 사라지면 부분도 함께 사라집니다.

실체화 (Realization) | 클래스가 인터페이스에 선언된 기능을 구현하여 정해진 계약을 충족합니다.

구별: 집합과 합성은 모두 전체·부분이지만, 부품의 독립적인 생존 여부가 핵심 차이입니다.

응집도

응집도는 높을수록 좋고 기능적 응집도가 가장 이상적이며 우연적 응집도가 가장 약합니다.

기능적 응집도 (Functional) | 모든 요소가 하나의 명확한 기능을 완성하기 위해 서로 밀접하게 협력합니다.

순차적 응집도 (Sequential) | 앞 작업의 출력이 다음 작업의 입력이 되어 단계 사이에 데이터가 흐릅니다.

교환적 응집도 (Communicational) | 같은 입력 데이터나 출력 결과를 다룬다는 이유로 여러 작업을 묶습니다.

절차적 응집도 (Procedural) | 데이터 흐름보다 정해진 실행 순서를 따라야 한다는 이유로 기능을 묶습니다.

시간적 응집도 (Temporal) | 기능은 달라도 초기화나 종료처럼 같은 시점에 실행된다는 이유로 모읍니다.

논리적 응집도 (Logical) | 성격이 비슷한 기능을 모으고 외부 선택값에 따라 특정 기능을 실행합니다.

우연적 응집도 (Coincidental) | 기능·데이터·실행 시점에 관련성이 없는 작업들을 편의상 한 모듈에 모읍니다.

구별: 순차적은 데이터가 단계 사이를 흐르고, 절차적은 실행 순서만 공유합니다.

결합도

결합도는 낮을수록 좋고 자료 결합도가 가장 바람직하며 내용 결합도가 가장 강합니다.

자료 결합도 (Data) | 필요한 단순 값만 인자로 전달하여 다른 모듈의 내부 구조에 덜 의존합니다.

스탬프 결합도 (Stamp) | 일부 값만 보내지 않고 구조체 전체를 전달하여 상대의 자료 구조에 의존합니다.

제어 결합도 (Control) | 상대의 처리 기능이나 실행 순서를 정하는 제어값을 전달하여 동작을 바꿉니다.

외부 결합도 (External) | 같은 외부 파일 형식이나 통신 규약·장치를 사용하여 외부 조건에 의존합니다.

공통 결합도 (Common) | 동일한 전역 데이터를 읽고 수정하여 한 모듈의 변경이 다른 모듈에 퍼집니다.

내용 결합도 (Content) | 다른 모듈의 내부 변수나 코드를 직접 참조하여 상대의 구현에 강하게 의존합니다.

구별: 자료는 필요한 값만, 스탬프는 묶인 자료 전체를 넘깁니다. 제어는 데이터가 아니라 상대의 동작을 바꿉니다.

아키텍처 5가지 뷰

Use-case View (유스케이스 뷰) | 사용자와 시스템의 기능 요구 및 주요 시나리오를 표현합니다.

Logical View (논리 뷰) | 클래스·객체·컴포넌트 등 시스템의 정적 구조와 책임 분할을 표현합니다.

Process View (프로세스 뷰) | 실행 중인 프로세스·스레드·통신·동시성·성능을 표현합니다.

Implementation View (구현 뷰) | 소스 코드·컴포넌트·패키지 등 구현 단위의 구성을 표현합니다.

Deployment View (배포 뷰) | 소프트웨어가 실행되는 서버·장치·네트워크의 물리적 배치를 표현합니다.

아키텍처 평가 모델

SAAM | 시나리오를 사용해 변경 용이성 등 품질 속성과 아키텍처의 영향을 분석합니다.

ATAM | 성능·가용성·보안·수정 용이성 같은 품질 속성 사이의 절충과 위험을 평가합니다.

CBAM | 아키텍처 전략의 비용과 품질 향상 효과를 함께 분석해 투자 우선순위를 정합니다.

ADR (아키텍처 결정 기록) | 중요한 아키텍처 선택과 그 배경·대안·결과를 문서로 남깁니다.

ARID (초기 설계 검토) | 초기 아키텍처의 구현 가능성과 적절성을 시나리오로 검토하는 평가 방법입니다.

XP의 다섯 가지 가치

의사소통 (Communication) | 고객과 개발자가 정보를 자주 공유해 오해를 줄입니다.

단순성 (Simplicity) | 현재 필요한 기능을 가장 단순한 설계로 구현합니다.

피드백 (Feedback) | 테스트와 고객 반응을 빠르게 받아 방향을 조정합니다.

용기 (Courage) | 문제를 드러내고 필요한 변경을 실행합니다.

존중 (Respect) | 팀원과 고객의 역할·기여·의견을 존중합니다.

인월·개발 기간·PERT 계산

개발 노력 (Man-Month, 인월) | LOC를 1인당 월간 생산성으로 나눈 총 작업량입니다.

예상 개발 기간 (Month) | 총 개발 노력을 투입 인원으로 나눈 개월 수입니다.

3점 기대치 (Three-point Estimate) | 낙관치·최빈치·비관치에 가중치를 주어 기대값을 구합니다.

PERT (프로그램 평가·검토 기법) | 작업 시간의 불확실성을 3점 추정으로 반영합니다.

CPM (주 공정법) | 선후 관계를 만족하는 경로 중 소요 시간이 가장 긴 경로를 찾습니다.

CCPM (중요 사슬 프로젝트 관리) | 작업 순서뿐 아니라 자원 제약과 버퍼를 고려합니다.

기능점수의 다섯 기능 유형

EI (외부 입력) | 경계 밖에서 들어와 내부 데이터를 유지하거나 동작을 바꾸는 기능입니다.

EO (외부 출력) | 계산·가공한 정보를 애플리케이션 경계 밖으로 내보냅니다.

EQ (외부 조회) | 입력 요청에 따라 데이터를 조회해 결과를 반환합니다.

ILF (내부 논리 파일) | 현재 애플리케이션이 유지·관리하는 논리 데이터 집합입니다.

EIF (외부 인터페이스 파일) | 다른 애플리케이션이 유지하며 현재 시스템은 참조하는 데이터입니다.

연계 데이터

XML | 태그와 속성으로 계층 구조와 데이터 의미를 표현하는 문서 형식입니다.

JSON | 키와 값으로 데이터를 간결하게 표현하는 교환 형식입니다.

CSV 형식 | 쉼표 같은 구분자로 각 필드를 나누어 표현하는 텍스트 데이터 형식입니다.

고정 길이 전문 | 각 필드가 정해진 문자 위치와 길이를 차지하는 연계 데이터 형식입니다.

데이터 매핑 | 송신 시스템의 필드와 수신 시스템의 필드 및 변환 규칙을 연결하는 작업입니다.

연계 모듈과 통신

동기 통신 | 요청자가 응답을 받을 때까지 다음 처리를 기다리는 통신 방식입니다.

비동기 통신 | 요청을 메시지나 큐에 맡기고 응답을 기다리지 않은 채 다음 작업을 수행하는 방식입니다.

연계 모듈 | 송신·수신·중계·변환 기능을 역할별로 나누어 시스템 간 통신을 담당하는 모듈입니다.

메시지 큐와 브로커 | 메시지를 임시 저장하고 적절한 소비자에게 전달하는 중간 구성 요소입니다.

테스트 수준과 유형

단위 테스트 | 함수나 클래스 등 작은 기능 단위를 분리하여 개별 동작이 올바른지 검사합니다.

통합 테스트 | 여러 모듈을 연결했을 때 데이터 전달과 인터페이스가 올바른지 검사합니다.

시스템 테스트 | 구성 요소를 합친 전체 시스템이 기능·성능 등 요구사항을 만족하는지 검사합니다.

인수 테스트 | 사용자·발주처의 관점에서 실제 업무와 계약한 요구를 충족하는지 확인합니다.

재테스트 (Retesting) | 수정 전에 실패한 테스트를 다시 실행하여 같은 오류가 해결됐는지 확인합니다.

회귀 테스트 (Regression Testing) | 변경 후 기존에 정상 동작하던 기능을 다시 검사하여 수정의 부작용을 찾아냅니다.

구별: 재테스트는 고친 결함 자체, 회귀 테스트는 변경의 영향이 퍼졌을 수 있는 다른 기능을 봅니다.

블랙박스 테스트

동치 분할 테스트 | 같은 결과가 예상되는 입력 범위를 여러 동등 클래스으로 나누고 대표값을 선택합니다.

경계값 분석 | 입력 허용 구간의 양 끝과 바로 안팎을 시험하여 경계에서 나는 오류를 찾습니다.

결정 테이블 테스트 | 조건 조합과 그에 따른 행동을 표로 만들어 규칙별 결과를 확인합니다.

원인-결과 그래프 | 입력 원인과 출력 결과 사이의 논리 관계를 그래프로 표현하여 테스트를 만듭니다.

오류 추정 (Error Guessing) | 경험과 과거 결함을 바탕으로 실수하기 쉬운 입력이나 처리 상황을 시험합니다.

페어와이즈 (Pairwise) | 매개변수 값의 모든 두 가지 조합을 포함하도록 테스트 수를 줄여 구성합니다.

화이트박스와 결함 관리

문장 커버리지 | 모든 실행 가능 문장을 한 번 이상 수행하며 판단식의 참·거짓은 따로 봅니다.

분기 커버리지 (Branch Coverage) | 각 판단식 전체가 참과 거짓이 되는 경로를 실행하여 분기별 동작을 검사합니다.

조건 커버리지 (Condition Coverage) | 복합 판단식을 구성하는 각 개별 조건의 참과 거짓이 모두 평가되도록 시험합니다.

조건/결정 커버리지 (C/DC) | 개별 조건과 전체 판단 결과에 참·거짓이 모두 나타나도록 두 기준을 함께 만족합니다.

변경 조건/결정 커버리지 (MC/DC) | 다른 조건은 고정하고 한 조건의 변화가 전체 판단 결과를 바꾸는지 확인합니다.

결함 생명주기 | 결함을 발견해 기록하고 분석·수정·재테스트·종료까지 상태로 관리합니다.

결함 심각도와 우선순위 | 심각도는 장애 영향의 크기이고 우선순위는 언제 먼저 처리할지를 나타냅니다.

테스트 오라클 | 테스트 결과가 올바른지 판단할 기준이나 기대 결과입니다.

구별: C/DC는 조건과 결정의 참·거짓을 모두 확인하고, MC/DC는 각 조건의 독립적인 영향까지 확인합니다. 단락 평가로 미실행된 조건은 구분하세요.

테스트 운영과 검증

검증과 확인 (Verification & Validation) | 검증은 개발 산출물이 정해진 명세와 절차에 맞는지, 확인은 사용자의 실제 요구를 만족하는지 평가합니다.

테스트 기반 | 테스트 케이스를 만들 때 근거로 사용하는 요구사항·설계서·코드·경험 자료입니다.

테스트 하네스 | 테스트 실행에 필요한 드라이버·스텁·테스트 데이터·도구를 묶은 환경입니다.

테스트 원칙 | 테스트는 결함의 존재를 보여줄 뿐 결함이 전혀 없음을 증명하지 못한다는 기본 원칙입니다.

회복 테스트 | 장애가 발생한 뒤 시스템과 데이터가 요구한 시간·상태로 복구되는지 확인합니다.

안전 테스트 | 비정상 접근이나 공격 상황에서 시스템이 자산을 보호하는지 확인합니다.

강도 테스트 (Stress Testing) | 정상 범위를 넘는 부하에서 시스템의 동작과 회복 능력을 확인합니다.

성능 테스트 | 응답 시간·처리량·자원 사용량 등 성능 요구사항을 확인합니다.

구조 테스트 | 프로그램 내부 구조와 논리 경로를 바탕으로 동작을 검증합니다.

병행 테스트 (Parallel Testing) | 기존 시스템과 새 시스템에 같은 입력을 주어 결과를 비교합니다.

요구사항 검토 | 요구사항 문서를 읽고 모호함·누락·충돌·검증 가능성을 확인합니다.

인스펙션 | 정해진 역할과 체크리스트를 사용해 산출물의 결함을 체계적으로 찾는 공식 검토입니다.

테스트 도구와 프레임워크 | 반복 실행과 결과 수집을 자동화하는 테스트 도구와 프레임워크입니다.

형상관리 활동

형상 식별 | 관리할 산출물을 형상 항목으로 정하고 이름·식별자·버전을 부여합니다.

형상 통제 | 형상 항목에 대한 변경을 요청·검토·승인하고 승인된 내용만 반영합니다.

형상 상태 기록 | 각 형상 항목의 현재 버전·변경 내역·승인 상태를 기록하고 보고합니다.

형상 감사 | 실제 산출물과 기록이 승인된 기준선 및 요구사항과 일치하는지 검토합니다.

베이스라인 (Baseline) | 검토와 승인을 거쳐 이후 변경의 기준이 되는 고정된 형상 상태입니다.

패키징 구성

패키징 명세서 | 제품의 구성 요소와 설치·운영에 필요한 정보를 정의한 문서입니다.

릴리스 노트 | 새 버전의 변경 사항과 설치·업데이트에 필요한 정보를 기록합니다.

의존성 관리 | 프로그램이 사용하는 외부 라이브러리와 실행 환경의 버전을 관리합니다.

설치 스크립트 | 프로그램과 필요한 구성 요소를 정해진 순서로 배치·설정하는 자동화입니다.

삭제 스크립트 | 프로그램 파일과 구성을 안전하게 제거하는 자동화입니다.

컨테이너·Docker | 애플리케이션과 실행 환경을 이미지로 묶어 격리된 컨테이너에서 실행하는 기술입니다.

테스트 케이스 구성

테스트 조건 | 테스트를 시작하기 전에 만족해야 할 상태와 전제를 정의합니다.

테스트 데이터 | 테스트 실행에 입력할 값과 경계·오류 사례를 준비합니다.

예상 결과 | 정해진 입력에 대해 시스템이 보여야 하는 구체적인 결과입니다.

판정 기준 | 실제 결과가 예상 결과와 얼마나 일치해야 성공인지 판단하는 규칙입니다.

실행 절차 | 테스트를 같은 방식으로 다시 수행할 수 있도록 작업 순서를 기록합니다.

실행 환경 | 테스트에 사용한 운영체제·브라우저·DB·네트워크 설정과 버전을 기록합니다.

테스트 오라클 | 실행 결과가 올바른지 판단하는 기준 또는 자동 판정 방법입니다.

커버리지와 통합 테스트

문장 커버리지 | 실행 가능한 각 문장이 테스트 중 한 번 이상 실행되었는지 측정합니다.

분기 커버리지 | 각 분기 판단의 참·거짓 결과를 모두 실행했는지 측정합니다.

조건 커버리지 | 복합 조건을 구성하는 각 조건의 참·거짓을 모두 확인합니다.

다중 조건 커버리지 | 복합 조건을 이루는 조건들의 가능한 참·거짓 조합을 시험합니다.

기본 경로 검사 | 제어 흐름 그래프에서 독립적인 실행 경로를 찾아 테스트합니다.

스텁 (Stub) | 테스트 중 아직 준비되지 않은 하위 호출 모듈을 대신하는 가짜 모듈입니다.

드라이버 (Driver) | 테스트 대상인 하위 모듈을 호출하는 상위 모듈을 대신하는 가짜 모듈입니다.

EAI 통합 방식 | Point-to-Point, Hub & Spoke, Message Bus, Hybrid는 시스템 사이의 연결 구조를 구분합니다.

소프트웨어 개발

처리량·응답 시간·경과 시간·사용률

처리량 (Throughput) | 단위 시간 동안 완료한 작업이나 요청의 양입니다.

응답 시간 (Response Time) | 요청한 시점부터 정한 응답을 받는 시점까지 걸린 시간입니다.

경과 시간 (Turnaround Time) | 작업을 의뢰한 시점부터 처리가 완료될 때까지의 시간입니다.

자원 사용률 (Resource Utilization) | 측정 구간의 CPU·메모리·회선 등 자원 사용 비율입니다.

보안 개발 방법론

Secure SDLC (보안 개발 생명주기) | 개발 전 과정에 보안 요구와 검증 활동을 통합합니다.

MS SDL (마이크로소프트 보안 개발 생명주기) | Microsoft가 정립한 개발 과정의 보안·프라이버시 관리 방식입니다.

Seven Touchpoints (세븐 터치포인츠) | 검증된 보안 실천 활동을 개발의 주요 접점에 결합합니다.

CLASP (경량 애플리케이션 보안 프로세스) | 역할과 활동을 중심으로 기존 개발 과정에 보안을 적용합니다.

배치 유형과 스케줄러

이벤트 배치 (Event-driven Batch) | 사전에 정한 사건이나 조건이 발생하면 실행합니다.

온디맨드 배치 (On-demand Batch) | 사용자나 운영자의 명시적인 요청으로 실행합니다.

정기 배치 (Scheduled Batch) | 정해진 시각이나 주기에 맞추어 반복 실행합니다.

Spring Batch (스프링 배치) | 대량 데이터의 읽기·처리·쓰기와 재시작을 지원하는 프레임워크입니다.

Quartz (쿼츠) | Job과 Trigger를 분리해 실행 시점을 관리하는 스케줄러입니다.

cron 표현식 (크론 표현식) | 정해진 필드로 반복 실행 시각을 표현합니다.

소프트웨어 개발

테스트 더블의 다섯 유형

Dummy (더미) | 매개변수 자리를 채우지만 시험에서는 사용하지 않는 객체입니다.

Stub (스텁) | 호출 시 미리 정한 응답을 제공하는 대역입니다.

Spy (스파이) | 호출 횟수와 인자 등 사용 내역을 기록하는 대역입니다.

Mock (목) | 예상한 호출·인자·순서 등의 상호작용을 검증하는 대역입니다.

Fake (페이크) | 실제로 동작하지만 시험에 맞게 단순화한 구현입니다.

테스트 오라클의 네 유형

참 오라클 (True Oracle) | 모든 시험 입력에 대해 정확한 기대 결과를 제공할 수 있습니다.

샘플링 오라클 (Sampling Oracle) | 선정한 일부 입력에 대해서만 정확한 기대 결과를 압니다.

휴리스틱 오라클 (Heuristic Oracle) | 확정 결과와 경험적 규칙·추정을 함께 활용합니다.

일관성 검사 오라클 (Consistent Oracle) | 기존 결과나 대응하는 실행 결과와의 일관성을 비교합니다.

테스트의 일곱 기본 원칙

결함의 존재 (Presence of Defects) | 테스트로 결함을 찾을 수 있지만 결함이 없음을 증명할 수는 없습니다.

완벽한 시험은 불가능 (Exhaustive Testing) | 현실적인 대부분의 시스템에서 모든 입력과 경로를 시험할 수 없습니다.

조기 시험 (Early Testing) | 개발 초기에 결함을 찾으면 재작업을 줄일 수 있습니다.

결함 집중 (Defect Clustering) | 일부 영역에 많은 결함이 집중되는 경향이 있습니다.

살충제 패러독스 (Pesticide Paradox) | 같은 시험만 반복하면 새로운 결함을 찾기 어려워집니다.

정황 의존 (Context Dependency) | 제품의 목적과 환경에 따라 필요한 시험이 달라집니다.

오류 부재의 궤변 (Absence-of-errors Fallacy) | 결함이 적어도 사용자 요구를 충족하지 못하면 성공한 제품이 아닙니다.

DRM 구성요소와 역할

콘텐츠 제공자 (Content Provider) | 저작권을 가진 콘텐츠를 제공하는 주체입니다.

패키저 (Packager) | 콘텐츠를 암호화하고 메타데이터와 함께 배포 단위로 묶습니다.

콘텐츠 분배자 (Content Distributor) | 보호된 콘텐츠를 사용자에게 유통합니다.

클리어링 하우스 (Clearing House) | 키·라이선스 발급과 권한 관련 관리를 수행합니다.

DRM 컨트롤러 (DRM Controller) | 사용자 환경에서 콘텐츠의 이용 권한을 검사·통제합니다.

보안 컨테이너 (Security Container) | 콘텐츠와 보호 정보를 안전한 유통 단위로 담습니다.

콘텐츠 소비자 (Content Consumer) | 정해진 이용 권한 아래 콘텐츠를 사용하는 주체입니다.

콘텐츠 식별·추적·패키징

워터마킹 (Digital Watermarking) | 콘텐츠에 저작권·소유권 식별 정보를 삽입합니다.

핑거프린팅 (Digital Fingerprinting) | 구매자나 배포본별 정보를 삽입해 유출 경로를 추적합니다.

DOI (디지털 객체 식별자) | 디지털 객체를 지속적으로 식별하는 식별자입니다.

권리 표현 (Rights Expression) | 사용 기간·횟수 등 허용 조건을 기계가 처리할 형태로 표현합니다.

사전 패키징 (Pre-packaging) | 요청이 오기 전에 보호된 배포 콘텐츠를 미리 생성합니다.

즉시 패키징 (On-the-fly Packaging) | 사용자의 요청 시점에 필요한 보호·배포 형식으로 포장합니다.

핵심 용어

데이터베이스 (Database) | 여러 사용자가 공동으로 이용하도록 통합하고 저장한 관련 데이터의 집합입니다.

DBMS (Database Management System) | 데이터를 정의·조작·제어하고 무결성·보안·동시성·복구를 관리하는 소프트웨어입니다.

스키마 (Schema) | 테이블과 속성·관계·제약 조건을 정의하며 현재 값과 구별되는 설계 명세입니다.

인스턴스 (Instance) | 특정 시점에 실제로 저장된 데이터의 상태로 행의 추가·수정·삭제에 따라 바뀝니다.

구별: 회원(회원번호, 이름, 등급)의 구조는 스키마이고 현재 저장된 회원 행들은 인스턴스입니다. 회원을 추가하면 인스턴스는 바뀌지만 구조가 같으면 스키마는 유지됩니다.

데이터베이스 구축

3단계 스키마

외부 스키마 (External Schema) | 개별 사용자나 응용 프로그램에 필요한 데이터만 보여 주는 부분 논리 구조입니다.

개념 스키마 (Conceptual Schema) | 조직 전체의 데이터와 관계·제약을 통합한 논리 구조로 사용자 관점을 조정합니다.

내부 스키마 (Internal Schema) | 파일 배치·레코드 형식·인덱스 등 저장 장치에 데이터를 배치하는 물리 구조입니다.

논리적 데이터 독립성 (Logical Data Independence) | 개념 스키마를 바꾸어도 외부 스키마와 응용 프로그램이 유지되도록 분리합니다.

물리적 데이터 독립성 (Physical Data Independence) | 내부 저장 구조를 바꾸어도 개념 스키마와 응용 프로그램이 유지되도록 분리합니다.

구별: 외부→개념 사이의 논리적 독립성과 개념→내부 사이의 물리적 독립성을 구분하세요. 인덱스 추가나 파일 배치 변경은 물리적 독립성의 예입니다.

데이터베이스 구축

DB 설계 절차

요구사항→개념적→논리적→물리적→구현 설계 순서로
구체화합니다.

요구사항 분석 | 업무에서 필요한
데이터·처리·조회·갱신·보안·성능 요구를 수집하고 명세화합니다.

개념적 설계 | 업무에서 중요한 엔터티·속성·관계를 DBMS와
무관한 개념 모델로 표현합니다.

논리적 설계 | 개념 모델을 릴레이션·키·무결성 제약 등 논리적
데이터 구조로 변환합니다.

물리적 설계 | 논리적 구조를 실제 저장·접근 방식으로 구현하기
위한 인덱스·파티션·저장 공간을 결정합니다.

구현 | 설계한 스키마와 데이터베이스 객체를 DBMS에 실제로
생성하고 데이터를 적재합니다.

관계형 구성 요소

릴레이션 (Relation) | 행과 열로 표현하는 데이터 집합으로 표 전체이며 이론상 중복 튜플을 허용하지 않습니다.

튜플 (Tuple) | 릴레이션의 한 행인 레코드로 하나의 대상을 나타내며 여러 속성 값을 함께 담습니다.

속성 (Attribute) | 릴레이션의 한 열인 필드로 대상의 특성을 나타내며 이름·자료형·도메인을 가집니다.

도메인 (Domain) | 한 속성에 허용되는 원자값의 집합으로 현재 저장된 값들의 목록과는 다릅니다.

카디널리티 (Cardinality) | 기수라고도 하며 릴레이션의 튜플 개수로 제목 행을 제외하고 데이터 행만 셉니다.

차수 (Degree) | 릴레이션의 속성 개수인 열 수로 데이터 행이 늘어도 구조가 같으면 유지됩니다.

구별: 회원(회원번호, 이름, 등급)에 회원 4명이 저장돼 있으면 카디널리티=4, 차수=3입니다. 등급의 허용 값이 일반·우수·VIP라면 그 집합이 도메인입니다. 모든 행을 삭제해도 구조가 같으면 차수는 3입니다. ER 관계의 카디널리티는 1:N 등 연결 수를 뜻합니다.

키의 종류

슈퍼키 (Super Key) | 행을 유일하게 식별하는 속성 집합으로 불필요한 속성이 포함되어도 됩니다.

후보키 (Candidate Key) | 행을 유일하게 식별하며 속성 하나라도 제거하면 유일성이 깨지는 최소 집합입니다.

기본키 (Primary Key) | 후보키에서 행의 대표 식별자로 선택하며 중복값과 NULL을 허용하지 않습니다.

외래키 (Foreign Key) | 다른 테이블의 기본키나 유일키를 참조하여 테이블 사이의 데이터 관계를 연결합니다.

구별: 후보키는 유일성+최소성을 모두 만족합니다. 기본키는 후보키 가운데 하나이며 NULL이 될 수 없습니다.

무결성 제약

개체 무결성 (Entity Integrity) | 각 행을 유일하게 식별하도록 기본키에 중복값이나 NULL이 들어가지 못합니다.

참조 무결성 (Referential Integrity) | NULL이 아닌 외래키 값이 부모 키에 존재하도록 하여 잘못된 참조를 막습니다.

도메인 무결성 (Domain Integrity) | 열의 값이 정해진 자료형·범위·허용 목록을 만족하도록 잘못된 입력을 막습니다.

ER 구성 요소

엔터티 (Entity) | 업무에서 독립적으로 관리하는 대상으로 고객·주문처럼 공통 속성의 개체를 묶습니다.

속성 (Attribute) | 엔터티가 가진 성질이나 상태를 표현하며 이름·주소처럼 대상의 특징을 담습니다.

관계 (Relationship) | 고객이 주문한다처럼 엔터티 사이에 맺어지는 업무상의 연관이나 사건을 나타냅니다.

식별자 (Identifier) | 엔터티의 각 인스턴스를 유일하게 구별하는 하나의 속성 또는 속성 조합입니다.

구별: Relation은 관계형 모델의 표이고 Relationship은 ER 모델에서 개체 사이의 연관입니다. 같은 "관계"라는 번역만 보고 혼동하지 마세요.

관계의 수와 참여

1:1 관계 | 양쪽의 한 개체가 상대편의 최대 한 개체와 연결되며 필수 여부는 따로 정합니다.

1:N 관계 | 한쪽의 한 개체가 상대편 여러 개체와 연결되며 일반적으로 N쪽에 외래키를 둡니다.

N:M 관계 | 양쪽 개체가 서로 여러 개체와 연결되므로 관계를 저장할 교차 테이블이 필요합니다.

참여 제약 (Participation) | 모든 개체가 관계에 반드시 참여해야 하는지 아니면 참여하지 않아도 되는지 정합니다.

구별: 고객 1명은 주문 여러 건과 연결되므로 고객:주문은 1:N입니다. 고객이 주문을 아직 안 했어도 된다면 선택 참여입니다. 여기의 카디널리티는 표의 현재 행 수가 아니라 관계의 연결 수를 뜻합니다.

함수 종속과 이상

함수 종속 (Functional Dependency) | 릴레이션의 속성 X의 값이 속성 Y의 값을 유일하게 결정하면 X→Y로 표현합니다.

완전 함수 종속 | 복합키 전체에 종속되고 키의 일부를 제거하면 더 이상 결정되지 않는 종속입니다.

부분 함수 종속 | 복합키의 일부 속성만으로도 비키 속성이 결정되는 종속입니다.

이행 함수 종속 | X→Y이고 Y→Z일 때 X→Z가 성립하며 비키 속성이 다른 비키 속성을 결정하는 종속입니다.

데이터베이스 구축

정규형 단계

제1정규형 (1NF) | 각 속성에 원자값 하나만 저장하여 반복되는 값이나 다중값 속성을 없앱니다.

제2정규형 (2NF) | 1NF에서 복합 후보키의 일부에만 종속된 속성을 분리하여 부분 종속을 없앱니다.

제3정규형 (3NF) | 2NF에서 키 아닌 속성을 통해 다른 속성이 키에 의존하는 이행 종속을 없앱니다.

BCNF | 모든 비자명한 함수 종속의 결정자가 슈퍼키가 되도록 하여 3NF를 강화합니다.

제4정규형 (4NF) | 독립적인 여러 다중값 사실을 별도 테이블로 분리하여 불필요한 조합을 없앱니다.

제5정규형 (5NF) | 분해 후 다시 조인해도 원래 결과를 유지하도록 비자명한 조인 종속을 제거합니다.

구별: 2NF는 부분 함수 종속, 3NF는 이행 함수 종속, BCNF는 슈퍼키가 아닌 결정자를 구별하는 것이 핵심입니다.

기본 관계대수 연산

셀렉션 (Selection, σ) | 조건을 만족하는 튜플만 선택하는 연산입니다.

프로젝션 (Projection, π) | 필요한 속성만 선택해 새로운 릴레이션을 만드는 연산입니다.

합집합 (Union, ∪) | 두 릴레이션 중 적어도 한쪽에 속한 튜플을 중복 없이 모읍니다.

교집합 (Intersection, ∩) | 두 릴레이션에 모두 속한 튜플만 선택합니다.

차집합 (Difference, -) | 첫 번째 릴레이션에는 있지만 두 번째 릴레이션에는 없는 튜플을 구합니다.

카티션 프로덕트 (×) | 두 릴레이션의 모든 튜플 조합을 만드는 연산입니다.

조회와 조건

SELECT | 테이블에서 조회할 열과 계산식을 지정하는 SQL 문입니다.

WHERE | 개별 행이 만족해야 하는 검색 조건을 지정합니다.

ORDER BY | 조회 결과를 하나 이상의 기준으로 정렬합니다.

LIKE | 문자열이 지정한 패턴에 맞는지 검사합니다.

IN | 값이 지정한 목록 또는 서브쿼리 결과에 속하는지 검사합니다.

BETWEEN | 값이 두 경계 사이에 있는지 검사하며 양 끝 경계도 포함합니다.

집계와 그룹화

COUNT | 행 또는 NULL이 아닌 특정 열의 개수를 세는 집계 함수입니다.

SUM | 지정한 숫자 열의 합계를 계산합니다.

AVG | 지정한 숫자 열의 평균을 계산합니다.

MAX | 지정한 열에서 가장 큰 값을 구합니다.

MIN | 지정한 열에서 가장 작은 값을 구합니다.

GROUP BY | 같은 값을 가진 행을 그룹으로 묶어 집계 결과를 계산합니다.

HAVING | GROUP BY로 만들어진 그룹이 만족해야 하는 조건을 지정합니다.

NULL과 표현식

NULL | 없거나 아직 알 수 없는 값의 상태로 일반 비교 대신 IS NULL로 확인합니다.

CASE | 조건에 따라 서로 다른 값을 반환하며 등급 표시나 조건별 계산에 사용합니다.

COALESCE·NVL | NULL을 대체할 값을 반환하며 COALESCE는 첫 번째 NULL 아닌 값을 선택합니다.

문자 함수 | 문자열 일부를 추출하거나 길이와 대소문자를 바꾸는 등 문자 데이터를 가공합니다.

날짜 함수 | 날짜의 구성 요소를 추출하거나 기간을 더하고 차이를 구해 날짜 데이터를 가공합니다.

구별: 값 = NULL과 NULL = NULL은 TRUE가 아닌 UNKNOWN입니다. COUNT(*)는 전체 행, COUNT(열)은 그 열이 NULL 아닌 행을 셉니다. 빈 문자열은 보통 NULL과 다르지만 Oracle은 빈 문자열을 NULL로 취급합니다.

집합 연산

UNION | 두 조회 결과를 합치고 동일한 행은 한 번만 남겨 전체 결과의 중복을 없앱니다.

UNION ALL | 두 조회 결과를 합치되 같은 행이 여러 번 나와도 제거하지 않고 모두 유지합니다.

INTERSECT | 양쪽 조회 결과에 공통으로 있는 행만 선택하고 한쪽에만 있는 행은 제외합니다.

EXCEPT·MINUS | 첫 조회에는 있지만 두 번째 조회에는 없는 행만 남겨 차집합 결과를 구합니다.

SQL 분류

DDL (Data Definition Language) | CREATE·ALTER·DROP으로 테이블 등의 구조를 생성·변경·삭제합니다.

DML (Data Manipulation Language) | 데이터를 SELECT로 조회하거나 INSERT·UPDATE 등으로 추가·수정합니다.

DCL (Data Control Language) | GRANT·REVOKE로 사용자 접근 권한을 부여하거나 기존 권한을 회수합니다.

TCL (Transaction Control Language) | COMMIT·ROLLBACK으로 트랜잭션의 변경 결과를 확정하거나 취소합니다.

제약 조건과 객체

제약 조건 (Constraint) | 데이터가 지켜야 하는 규칙을 DBMS가 검사하도록 정의한 조건입니다.

뷰 (View) | 하나 이상의 테이블에 대한 SELECT 결과를 가상 테이블처럼 제공하는 객체입니다.

인덱스 (Index) | 검색 조건과 정렬에 사용할 수 있는 별도의 접근 경로입니다.

저장 프로시저 (Procedure) | DB에 저장된 SQL 명령과 절차를 명시적으로 호출해 실행합니다.

저장 함수 (Function) | 입력값을 받아 계산하거나 조회한 결과를 반환하는 DB 객체입니다.

트리거 (Trigger) | 테이블의 INSERT·UPDATE·DELETE 같은 사건이 발생할 때 자동으로 실행되는 DB 객체입니다.

트랜잭션과 ACID

원자성 (Atomicity) | 트랜잭션의 모든 연산이 성공하거나 모두 취소되어야 한다는 성질입니다.

일관성 (Consistency) | 트랜잭션 전후에 데이터베이스가 정의된 무결성 제약과 규칙을 만족해야 한다는 성질입니다.

격리성 (Isolation) | 동시에 실행되는 트랜잭션이 서로의 중간 결과를 보지 못하도록 하는 성질입니다.

지속성 (Durability) | 커밋이 완료된 트랜잭션의 결과가 장애 후에도 보존되는 성질입니다.

분산 DB 투명성의 구분

위치 투명성 (Location Transparency) | 데이터가 저장된 물리적 위치를 몰라도 접근할 수 있습니다.

분할 투명성 (Fragmentation Transparency) | 하나의 논리 데이터를 여러 단편으로 나눈 사실을 숨깁니다.

복제 투명성 (Replication Transparency) | 여러 데이터 사본을 하나의 논리 데이터처럼 사용합니다.

병행 투명성 (Concurrency Transparency) | 여러 트랜잭션의 동시 실행에도 일관된 결과를 유지합니다.

장애 투명성 (Failure Transparency) | 일부 장애를 시스템이 처리해 데이터의 일관된 사용을 지원합니다.

이주 투명성 (Migration Transparency) | 데이터나 자원이 이동해도 사용 방법·이름을 유지합니다.

병행 제어 기법 비교

낙관적 검증 (Optimistic Validation) | 일단 작업하고 반영 전에 충돌을 검사합니다.

타임스탬프 순서 제어 (Timestamp Ordering) | 트랜잭션의 논리적 시간 순서를 기준으로 충돌 연산을 제어합니다.

MVCC (다중 버전 동시성 제어) | 데이터의 여러 버전을 유지해 읽기 일관성을 제공합니다.

큰 로킹 단위 (Coarse Granularity) | 테이블 등 넓은 범위를 적은 수의 잠금으로 제어합니다.

작은 로킹 단위 (Fine Granularity) | 행 등 좁은 범위를 개별 잠금으로 제어합니다.

회복 기법의 갱신 시점

즉시 갱신 (Immediate Update) | 트랜잭션 확정 전에 변경 페이지가 저장장치에 반영될 수 있습니다.

지연 갱신 (Deferred Update) | 확정 전에는 변경을 실제 DB에 반영하지 않는 고전적 회복 방식입니다.

그림자 페이징 (Shadow Paging) | 원본 페이지 테이블을 보존하고 변경 페이지를 별도로 관리합니다.

체크포인트 (Checkpoint) | 회복 작업의 시작 범위를 줄이기 위해 상태를 기록하는 지점입니다.

API·플러그인·TDE·하이브리드

API 방식 (Application API) | 애플리케이션이 암호 모듈을 호출해 암복호화합니다.

플러그인 방식 (Plug-in) | DB 서버에 설치한 암호 모듈을 통해 처리합니다.

TDE (투명한 데이터 암호화) | DBMS가 저장 데이터의 암복호화를 내부적으로 수행합니다.

하이브리드 방식 (Hybrid) | 애플리케이션과 DB 계층의 방식을 조합합니다.

NoSQL의 네 가지 저장 모델

Key-Value (키-값) | 고유 키와 연결된 값의 형태로 저장합니다.

Document (문서) | JSON과 유사한 문서 단위로 자료를 저장합니다.

Wide-column (와이드 컬럼) | 행 키와 컬럼 패밀리를 중심으로 데이터를 구성합니다.

Graph (그래프) | 노드·관계와 그 속성으로 데이터를 표현합니다.

연산자와 우선순위

산술 연산자 | 덧셈·뺄셈·곱셈·나눗셈·나머지로 값을 계산합니다.

대입 연산자 | 계산한 값을 변수에 저장하거나 복합 대입으로 계산과 저장을 함께 수행합니다.

증감 연산자 | ++와 --는 값을 1만큼 변경하며 전위·후위 위치에 따라 식에 사용되는 시점이 달라집니다.

비교 연산자 | 두 값의 크기나 동등 여부를 비교해 참·거짓 결과를 만듭니다.

논리 연산자 | 참·거짓 조건을 AND·OR·NOT으로 결합합니다.

비트 연산자 | 정수의 이진수 비트마다 AND·OR·XOR·반전 또는 시프트를 수행합니다.

삼항 연산자 | 조건에 따라 두 표현식 중 하나의 결과를 선택합니다.

조건문과 분기

if·else if·else | 조건을 위에서부터 검사해 처음 참인 블록 또는 else 블록을 실행합니다.

switch·case | 하나의 식을 case 값과 비교해 일치하는 구간을 실행하는 분기문입니다.

삼항 조건식 | 조건에 따라 두 표현식 중 하나를 선택해 값을 반환하는 간결한 조건문입니다.

반복문과 흐름 제어

for 문 | 반복 횟수나 인덱스의 시작·종료·증감을 한 곳에 표현하는 반복문입니다.

while | 조건이 참인 동안 본문을 반복하며 실행 전에 조건을 먼저 검사합니다.

do-while | 본문을 먼저 실행하고 마지막에 반복 조건을 검사합니다.

break | 가장 가까운 반복문 또는 switch를 즉시 종료합니다.

continue | 현재 반복의 남은 문장을 건너뛰고 다음 반복으로 넘어갑니다.

함수와 재귀

값에 의한 전달 | 호출 시 인자의 값이 매개변수에 복사되어 함수 내부 변경이 원본 변수에 직접 반영되지 않는 방식입니다.

C 주소 전달 | C 함수에 변수의 주소를 포인터 매개변수로 전달해 원본 저장 위치를 다룹니다.

객체 참조 전달 | Java·Python에서 객체를 가리키는 참조를 매개변수로 전달합니다.

재귀 함수 | 함수가 자기 자신을 호출해 입력을 줄여 가며 문제를 해결하는 함수입니다.

호출 스택 | 함수 호출마다 매개변수·지역 변수·복귀 위치를 저장하는 실행 영역입니다.

배열과 인덱스

1차원 배열 | 같은 자료형의 값을 일렬로 저장하고 인덱스로 접근하는 자료구조입니다.

배열 복사와 교환 | 배열 요소를 임시 변수에 저장하거나 언어의 복사 기능으로 다른 배열에 옮기는 작업입니다.

역순 접근 | 배열이나 문자열을 마지막 인덱스부터 처음까지 거꾸로 읽습니다.

대칭 접근 | 앞쪽 i와 뒤쪽 n-1-i를 한 쌍으로 비교하거나 교환합니다.

문자열 처리

C 문자열과 널 문자 | C에서 문자열은 문자 배열 끝에 ₩0이 들어 있는 형태로 표현됩니다.

문자열 슬라이싱 | 문자열이나 리스트의 시작·끝·간격을 지정해 일부를 추출하는 문법입니다.

문자열 연결 | 여러 문자열을 하나로 이어 붙이는 작업입니다.

문자열 분리 | 구분자를 기준으로 문자열을 여러 부분으로 나누는 작업입니다.

문자열 비교 | 문자열의 내용 또는 참조를 비교하는 연산입니다.

포인터 핵심

주소 연산자 (&) | 변수의 메모리 주소를 구하는 연산자입니다.

역참조 연산자 (*) | 포인터가 가리키는 주소의 실제 값을 읽거나 수정하는 연산자입니다.

포인터 연산 | 포인터에 정수를 더하거나 빼 자료형 크기만큼 다음 요소 주소로 이동하는 연산입니다.

NULL 포인터 | 유효한 객체를 가리키지 않는 포인터 상태를 표현하는 값입니다.

포인터와 함수·배열

값에 의한 전달 | 함수 호출 시 변수의 값만 복사해 매개변수에 전달하는 방식입니다.

주소에 의한 전달 | 변수의 주소를 포인터로 전달해 함수가 원본 메모리에 접근하는 방식입니다.

배열 이름과 포인터 | 배열 이름은 대부분의 식에서 첫 번째 요소를 가리키는 주소처럼 사용됩니다.

문자 포인터와 문자 배열 | 문자 배열은 수정 가능한 저장 공간이고 문자열 리터럴을 가리키는 포인터는 수정하면 안 되는 경우가 많습니다.

Python 컬렉션

list (리스트) | 순서와 중복값을 유지하며 생성 후 요소를 추가하거나 수정할 수 있는 시퀀스입니다.

tuple (튜플) | 순서와 중복값을 유지하지만 생성 후 요소 참조를 바꿀 수 없는 불변 시퀀스입니다.

set (집합) | 원소를 중복 저장하지 않으며 위치보다 원소 포함 여부와 집합 연산을 다룹니다.

dict (딕셔너리) | 유일한 키에 값을 연결하며 같은 키로 저장하면 기존 값은 새 값으로 바뀝니다.

인덱싱과 슬라이싱

인덱싱 | 하나의 위치 번호로 문자열·리스트·튜플의 요소를 가져오는 연산입니다.

슬라이싱 | start부터 end 직전까지 step 간격으로 여러 요소를 추출하는 문법입니다.

언패킹 | 컬렉션의 요소를 여러 변수에 순서대로 나누어 대입하는 문법입니다.

함수와 변수 범위

지역 변수 (Local) | 함수 호출 중에 만들어지고 해당 함수 범위에서만 접근할 수 있는 변수입니다.

전역 변수와 global | 모듈 또는 함수 밖에 선언되어 여러 함수에서 참조할 수 있는 변수와 이를 함수 안에서 재할당하는 선언입니다.

기본 인자와 반환 | 인자를 생략했을 때 사용할 기본값과 함수가 호출자에게 돌려주는 결과입니다.

복사와 참조

대입과 참조 공유 | 새 변수도 같은 객체를 가리키므로 한쪽에서 객체를 변경하면 다른 쪽에도 보입니다.

얕은 복사 (Shallow Copy) | 바깥 객체만 새로 만들고 중첩 객체는 공유하므로 내부 변경이 원본에도 반영됩니다.

깊은 복사 (Deep Copy) | 중첩 객체까지 재귀적으로 복제하여 복사본의 내부 변경이 원본과 분리됩니다.

구별: 중첩 리스트를 수정했을 때 얕은 복사본도 변하면, 안쪽 객체를 공유하고 있다는 뜻입니다.

클래스와 생성자

클래스와 객체 | 클래스는 속성과 메서드의 설계도이고 객체는 그 클래스로 만들어진 실제 인스턴스입니다.

생성자 (Constructor) | 객체가 생성될 때 상태를 초기화하는 특별한 메서드입니다.

this | 현재 객체의 필드·메서드나 현재 클래스의 다른 생성자를 참조하는 Java 키워드입니다.

super | 부모 클래스의 생성자·필드·메서드를 참조하는 Java 키워드입니다.

상속과 다형성

상속 (Inheritance) | 자식 클래스가 부모 클래스의 멤버와 규칙을 물려받아 확장하는 기능입니다.

오버라이딩 (Overriding) | 자식 클래스가 부모의 같은 메서드를 자신의 동작으로 재정의하는 방식입니다.

다형성 (Polymorphism) | 하나의 부모 타입이나 인터페이스로 여러 구체 객체를 다루는 성질입니다.

업캐스팅 (Upcasting) | 자식 클래스 객체를 부모 클래스 타입의 참조로 다룹니다.

다운캐스팅 (Downcasting) | 부모 타입 참조를 실제 자식 클래스 타입으로 변환합니다.

연산자와 분기 문법

증감 연산자 | ++, --로 변수 값을 1 늘리거나 줄입니다.

복합 대입 연산자 | +=, -=, *= 등으로 계산 결과를 같은 변수에 다시 저장합니다.

비트 논리 연산 | 정수의 이진 표현을 자리별로 AND·OR·XOR합니다.

비트 시프트 연산 | 정수의 비트 패턴을 왼쪽·오른쪽으로 이동합니다.

switch fall-through | case에 진입한 뒤 break가 없으면 아래 case의 문장도 이어서 실행됩니다.

배열 기본값과 길이 | Java 배열은 생성 시 자료형별 기본값으로 초기화되고 length로 크기를 확인합니다.

예외 흐름

try·catch | 예외가 발생할 수 있는 코드를 try에 두고 종류별 처리 코드를 catch에 작성합니다.

finally | 예외 발생 여부나 catch 실행 여부와 관계없이 마무리 작업을 수행하는 블록입니다.

예외 상속과 catch 순서 | 구체적인 예외는 넓은 상위 예외 타입에 포함될 수 있습니다.

throw | Java 코드에서 예외 객체를 직접 발생시키는 키워드입니다.

throws | Java 메서드 선언부에 발생 가능 예외를 명시해 호출자에게 처리를 맡기는 키워드입니다.

오버로딩과 오버라이딩

오버로딩 (Overloading) | 이름은 같지만 매개변수 타입·개수가 달라 전달한 인자에 맞는 메서드를 고릅니다.

오버라이딩 (Overriding) | 상속받은 메서드를 같은 시그니처로 다시 정의하여 하위 클래스의 동작을 바꿉니다.

정적 바인딩 (Static Binding) | 선언된 참조 타입과 인자 타입을 보고 컴파일 시 호출할 시그니처를 선택합니다.

동적 바인딩 (Dynamic Binding) | 실행 중 실제 객체의 타입을 보고 재정의된 인스턴스 메서드 구현을 선택합니다.

구별: 오버로딩은 인자 타입·개수, 오버라이딩은 실제 객체 타입이 핵심 단서입니다.

프로그래밍 언어 활용

Java List·Set·Map 구현

ArrayList (배열 리스트) | 인덱스 순서와 중복 원소를 유지하는 동적 배열 기반 List입니다.

LinkedList (연결 리스트) | 노드의 연결로 순서를 유지하며 중복을 허용하는 List입니다.

HashSet (해시 집합) | 중복을 허용하지 않으며 원소의 순회 순서를 보장하지 않습니다.

HashMap (해시 맵) | 고유한 키에 값을 연결하며 순회 순서를 보장하지 않습니다.

시간 지표 공식

완료 시각을 찾은 뒤 반환=완료-도착, 대기=반환-실행, 응답=첫 실행-도착을 적용합니다.

반환시간 (Turnaround Time) | 프로세스가 도착한 순간부터 모든 실행을 마치기까지 걸린 시간입니다.

대기시간 (Waiting Time) | 프로세스가 준비 큐에서 CPU를 기다린 시간의 합입니다.

응답시간 (Response Time) | 프로세스가 도착한 뒤 처음으로 CPU를 받기까지 걸린 시간입니다.

평균 시간 | 각 프로세스에서 구한 같은 종류의 시간을 모두 더해 프로세스 수로 나눈 값입니다.

알고리즘별 시간축

FCFS | 먼저 도착한 프로세스를 먼저 실행하는 비선점형 스케줄링입니다.

SJF | 현재 준비된 작업 중 실행 시간이 가장 짧은 프로세스를 선택하는 비선점형 스케줄링입니다.

SRTF | 현재 작업의 남은 시간과 새로 도착한 작업의 실행 시간을 비교해 더 짧은 작업을 실행합니다.

Round Robin (RR) | 준비 큐의 각 프로세스에 시간 할당량만큼 CPU를 주고 미완료 작업은 큐 뒤로 보냅니다.

HRN | 대기시간을 반영한 응답률이 가장 높은 프로세스를 선택하는 비선점 방식입니다.

비선점형 시간축 계산 예시

공통 계산 공식 | 완료 시각 C, 도착 A, 실행 B일 때 반환시간 T=C-A, 대기시간 W=T-B입니다.

FCFS (선착순) | 도착 순서로 P1 0~6 → P2 6~9 → P3 9~10을 실행합니다.

SJF (최단 작업 우선) | 시각 0에는 P1만 도착했으므로 P1 0~6, 이후 짧은 P3 6~7, P2 7~10입니다.

HRN (최고 응답률 우선) | 시각 6에 응답률 R=(대기시간+실행시간)/실행시간이 가장 큰 작업을 선택합니다.

선점형 시간축 계산 예시

SRTF (최단 잔여시간 우선) | 시각 1에 P2(3)가 P1의 잔여 5보다 짧아 선점합니다.

RR (시간 할당량 2) | 각 작업을 최대 2만큼 실행하고 미완료 작업은 준비 큐 뒤로 보냅니다.

응답시간·반환시간 검산 | 응답시간=첫 실행-도착, 반환시간=완료-도착입니다.

프리픽스 변환

부분 마스크는 128·192·224·240·248·252·254, 블록 크기는 256에서 부분 마스크를 뺍니다.

/24~30 빠른 표 | /24부터 마지막 옥텟의 마스크는 0·128·192·224·240·248·252 순서입니다.

호스트 비트 | IPv4 전체 32비트에서 프리픽스 길이를 뺀 값입니다.

변화하는 옥텟 | 서브넷 경계가 실제로 바뀌는 마스크의 부분 옥텟입니다.

블록 크기 | 변화하는 옥텟에서 한 서브넷이 차지하는 주소 간격입니다.

주소 범위 계산

네트워크 주소 | 호스트 비트를 모두 0으로 만든 해당 주소 블록의 첫 번째 주소입니다.

브로드캐스트 주소 | 호스트 비트를 모두 1로 만든 해당 주소 블록의 마지막 주소입니다.

사용 가능 범위 | 일반적인 서브넷에서 네트워크와 브로드캐스트를 제외한 주소 범위입니다.

같은 네트워크 판별 | 두 IP에 같은 마스크를 적용했을 때 네트워크 주소가 같으면 같은 서브넷입니다.

브로드캐스트 주소 계산 공식·예시

비트 계산 공식 | IPv4 네트워크 주소는 IP AND 서브넷 마스크, 브로드캐스트 주소는 네트워크 OR (32비트 마스크의 반전)입니다.

/26 마지막 옥텟 예시 | 192.168.10.77/26은 블록 크기 256-192=64로 64~127 구간에 속합니다.

/22 세 번째 옥텟 예시 | 172.16.9.130/22는 마스크 255.255.252.0으로 세 번째 옥텟이 4씩 변합니다.

브로드캐스트 예외·구분 | 255.255.255.255는 로컬 한정 브로드캐스트이고 192.168.10.127은 위 /26 서브넷의 지향 브로드캐스트입니다.

정보시스템 구축 관리

서브넷 분할 계산 공식·실기형 예시

문제 조건 확인 | 원래 대역 10.20.4.64/27을 2분할하고, 10.20.4.82가 속한 서브넷의 프리픽스와 브로드캐스트 주소를 구합니다.

빌릴 비트·새 마스크 | 2개로 분할하므로 2^s≥2인 최소 s=1을 빌려 /28로 만듭니다.

IP가 속한 구간 | 10.20.4.82의 마지막 옥텟 82는 둘째 구간 80~95에 있습니다.

최종 답안과 검산 | 정답은 프리픽스 /28과 브로드캐스트 주소 10.20.4.95입니다.

표 작성 원칙

페이지 적중 (Hit) | 요청한 페이지가 이미 프레임에 있어 추가 적재가 필요 없는 상태입니다.
페이지 부재 (Fault) | 요청한 페이지가 프레임에 없어 보조기억장치에서 가져와야 하는 상태입니다.
초기 적재와 교체 | 초기 적재는 빈 프레임을 채우는 작업이고 교체는 기존 페이지를 내보내고 새 페이지를 넣는 작업입니다.
프레임 표 | 각 참조 직후 프레임의 전체 상태를 열 단위로 기록한 표입니다.

교체 알고리즘

FIFO | 프레임에 가장 먼저 들어온 페이지를 가장 먼저 교체합니다.

LRU | 현재 시점에서 가장 오랫동안 참조되지 않은 페이지를 교체합니다.

LFU | 누적 참조 횟수가 가장 적은 페이지를 교체합니다.

OPT | 앞으로 가장 늦게 다시 사용되거나 다시 사용되지 않는 페이지를 교체합니다.

NUR/NRU (최근 미사용) | 참조 비트 R과 변경 비트 M으로 페이지를 분류해 교체 대상을 고릅니다.

SCR (2차 기회·Second Chance) | FIFO 순서에서 참조 비트가 1인 페이지에 한 번 더 기회를 줍니다.

라우팅 프로토콜

RIP | 같은 AS 내부에서 거치는 라우터의 홉 수를 비교하여 가장 적은 경로를 고릅니다.

OSPF | 같은 AS 내부의 링크 상태를 공유하여 비용의 합이 가장 작은 경로를 구합니다.

BGP | 다른 AS 사이에서 경로 속성을 교환하며 조직의 정책을 반영해 경로를 고릅니다.

구별: RIP의 홉 수가 적은 경로와 OSPF의 비용이 낮은 경로는 같은 경로가 아닐 수 있습니다. BGP는 두 방식의 단순 최단 경로 알고리즘이 아닙니다.

OSI와 TCP/IP

물리·링크·네트워크·전송·세션·표현·응용 순서로 아래에서 위로 올라갑니다.

물리 계층 (1계층) | 비트를 전기·광·무선 신호로 바꾸어 전송 매체로 전달합니다.

데이터링크 계층 (2계층) | 같은 링크의 노드 사이에서 프레임을 전달하고 MAC 주소와 오류 검출을 다룹니다.

네트워크 계층 | 서로 다른 네트워크 사이에서 IP 주소를 이용해 패킷의 경로를 선택하고 전달합니다.

전송 계층 | 송신 프로세스와 수신 프로세스 사이의 종단 간 전송, 포트, 흐름·오류·혼잡 제어를 담당합니다.

세션 계층 (5계층) | 응용 프로그램 사이의 대화 연결을 설정·유지·종료하고 대화 흐름을 관리합니다.

표현 계층 (6계층) | 응용 데이터의 표현 형식을 맞추고 인코딩·압축·암호화 같은 변환을 다룹니다.

응용 계층 (7계층) | 사용자 응용 프로그램에 네트워크 서비스를 제공하는 최상위 계층입니다.

TCP와 UDP

TCP | 연결을 맺고 순서 번호와 재전송으로 데이터가 빠짐없이 순서대로 도착하게 합니다.

UDP | 연결 없이 보내 지연을 줄이지만 유실 복구나 순서 보장은 자체 제공하지 않습니다.

포트 (Port) | 한 호스트 안에서 통신할 서비스나 프로세스를 구분하기 위해 사용하는 숫자입니다.

소켓 (Socket) | IP와 포트 등으로 구성된 통신 끝점으로 데이터를 주고받을 대상을 나타냅니다.

응용 프로토콜

HTTP | 웹 클라이언트와 서버가 요청·응답으로 자원을 주고받는 응용 계층 프로토콜입니다.

HTTPS | HTTP 통신을 TLS로 보호해 전송 구간을 암호화하는 방식입니다.

DNS | 도메인 이름에 대응하는 IP 주소 등을 조회하는 이름 해석 서비스입니다.

DHCP | 호스트에 IP 주소와 기본 게이트웨이·DNS 서버 등의 네트워크 설정을 자동으로 제공합니다.

FTP | 클라이언트와 서버 사이에 파일을 전송하는 응용 계층 프로토콜입니다.

SMTP | 전자우편을 보내거나 메일 서버 사이에 전달하는 프로토콜입니다.

POP3 | 메일 서버에서 전자우편을 받아오는 수신 프로토콜입니다.

IMAP | 메일을 서버에 두고 폴더·읽음 상태 등을 여러 장치와 동기화하는 수신 프로토콜입니다.

RTP | 실시간 음성·영상 데이터를 전달하는 프로토콜입니다.

RTCP | RTP 전송의 품질과 제어 정보를 교환하는 프로토콜입니다.

AAA와 공격 분류

인증 (Authentication) | 사용자가 주장하는 신원이 실제 본인인지 확인합니다.

인가 (Authorization) | 인증된 사용자가 특정 자원에 접근하거나 작업할 권한이 있는지 판단합니다.

책임 추적 (Accountability) | 누가 언제 어떤 자원에 접근하고 작업했는지 기록·추적합니다.

DoS (서비스 거부) | 서버나 네트워크의 자원을 고갈시켜 정상 사용자의 접속을 막는 공격입니다.

DDoS (분산 서비스 거부) | 여러 분산된 공격원이 동시에 대상의 가용성을 떨어뜨리는 공격입니다.

스푸핑 (Spoofing) | 다른 주체의 IP·MAC·이메일 주소나 신원을 사칭하는 공격입니다.

스니핑 (Sniffing) | 네트워크를 오가는 패킷을 가로채거나 관찰해 정보를 수집하는 행위입니다.

피싱 (Phishing) | 가짜 메시지나 사이트로 사용자를 속여 비밀번호·금융 정보를 입력하게 하는 공격입니다.

파밍 (Pharming) | 주소 입력이나 이름 해석 과정을 조작해 정상 주소를 입력해도 가짜 사이트로 보내는 공격입니다.

네트워크·웹 공격

플러딩 (Flooding) | 대량의 패킷이나 요청을 보내 네트워크·서버 자원을 소진시키는 공격 방식입니다.

스머핑 (Smurfing) | 피해자 IP를 출발지로 위조한 ICMP 요청을 브로드캐스트 주소에 보내 응답을 피해자에게 집중시키는 공격입니다.

랜드 공격 (LAND) | 패킷의 출발지와 목적지 IP·포트를 피해자 자신의 값으로 위조해 처리에 부담을 주는 공격입니다.

XSS (Cross-Site Scripting) | 공격자가 입력한 스크립트가 다른 사용자의 브라우저에서 실행되도록 하는 웹 공격입니다.

SQL Injection | 입력값에 SQL 구문을 삽입해 인증 우회·정보 조회·데이터 변경을 시도하는 공격입니다.

무차별 대입 공격 (Brute Force) | 가능한 비밀번호 조합을 차례로 시도해 인증을 통과하려는 공격입니다.

사전 공격 (Dictionary Attack) | 자주 쓰는 단어나 유출된 비밀번호 목록을 우선 시도하는 공격입니다.

공격 유형

Session Hijacking (세션 하이재킹) | 인증된 사용자의 세션을 탈취하여 다시 비밀번호를 확인하지 않고 권한을 악용합니다.

Watering Hole (워터링 홀) | 공격 대상이 자주 방문하는 사이트를 감염시켜 그 사이트의 방문자를 공격합니다.

스케어웨어 (Scareware) | 가짜 감염 경고로 공포심을 유발하여 결제나 악성 프로그램 설치를 유도합니다.

이블 트윈 (Evil Twin) | 정상 무선 AP처럼 위장한 가짜 AP로 접속을 유도하여 통신이나 정보를 노립니다.

구별: 스케어웨어는 가짜 경고로 속이는 방식, 랜섬웨어는 실제 암호화·접근 차단 후 금전을 요구하는 방식입니다.

인증과 관리

OTP (일회용 비밀번호) | 한 번 사용하거나 짧은 시간만 유효한 비밀번호로 인증값의 재사용을 줄입니다.

OAuth (권한 위임) | 비밀번호를 제3자 서비스에 넘기지 않고 토큰으로 자원 접근 권한을 위임합니다.

SSO (통합 인증) | 한 번 인증한 사용자가 여러 관련 서비스에 다시 로그인하지 않고 접근합니다.

ISMS (정보보호 관리체계) | 조직의 정보보호 정책과 위험·보호대책을 지속적으로 운영하고 점검하는 체계입니다.

구별: OAuth는 권한 위임 표준이며 그 자체가 로그인 인증 표준은 아닙니다. OpenID Connect는 OAuth 2.0 위에 인증 계층을 추가합니다.

악성코드 종류와 구별

바이러스 (Virus) | 다른 프로그램이나 파일에 붙어 복제하며 감염된 숙주가 실행될 때 활성화됩니다.

웜 (Worm) | 숙주 프로그램에 붙지 않고 독립적으로 복제되어 다른 시스템으로 전파됩니다.

트로이목마 (Trojan Horse) | 정상 프로그램으로 위장해 숨은 악성 기능을 실행하며 자체 복제가 핵심은 아닙니다.

랜섬웨어 (Ransomware) | 파일 암호화나 접근 차단으로 피해를 주고 복구 대가로 금전을 요구합니다.

스파이웨어 (Spyware) | 사용자 모르게 활동과 정보를 수집해 외부로 전달하는 감시 소프트웨어입니다.

루트킷 (Rootkit) | 공격자의 파일이나 프로세스 등을 숨겨 보안 검사에서 발견하기 어렵게 합니다.

키로거 (Keylogger) | 키보드 입력을 기록하는 도구로 악용하면 비밀번호 등 입력 정보를 훔칩니다.

백도어 (Backdoor) | 정상적인 인증이나 보안 검사를 우회해 시스템에 접근하는 숨겨진 경로입니다.

논리 폭탄 (Logic Bomb) | 지정한 날짜나 사건 등 조건이 충족될 때 숨겨 둔 악성 동작을 실행합니다.

구별: 바이러스·웜은 복제·전파 방식, 트로이목마는 위장, 랜섬웨어는 피해 행동에 주목한 구분입니다. 한 악성코드가 여러 특성을 함께 가질 수 있습니다.

암호 방식

대칭키 암호 | 같은 비밀키로 암호화·복호화하여 빠르게 처리하지만 안전한 키 공유가 필요합니다.

공개키 암호 | 공개키와 개인키를 한 쌍으로 사용하여 공개된 키와 비밀키의 역할을 나눕니다.

하이브리드 암호 | 세션키는 공개키 방식으로 전달하고 본문은 빠른 대칭키 방식으로 암호화합니다.

보안 프로토콜

SSL/TLS | 웹 등 응용 프로토콜의 통신 구간을 암호화하고 서버 인증과 무결성을 제공하는 보안 프로토콜입니다.

IPSec | IP 계층에서 패킷을 인증·암호화해 네트워크 구간을 보호하는 프로토콜 모음입니다.

SSH | 원격 호스트 접속과 명령 실행, 파일 전송을 암호화하는 보안 프로토콜입니다.

WPA | WEP의 취약점을 보완하기 위해 도입된 무선 LAN 보안 규격입니다.

WPA2 | AES 기반 CCMP를 채택한 무선 LAN 보안 규격입니다.

WPA3 | 향상된 인증·암호화 보호를 제공하는 무선 LAN 보안 규격입니다.

접근 통제 모델

DAC (임의 접근 통제) | 자원의 소유자가 허용 대상을 정하고 자신의 판단에 따라 접근 권한을 부여합니다.

MAC (강제 접근 통제) | 중앙의 보안 등급과 정책이 접근을 결정하므로 소유자가 임의로 허용할 수 없습니다.

RBAC (역할 기반 접근 통제) | 직무·역할에 권한을 묶어 부여하고 사용자가 맡은 역할을 기준으로 접근을 정합니다.

ABAC (속성 기반 접근 통제) | 사용자·자원·시간·위치 등 속성을 정책 규칙과 비교하여 현재 접근을 결정합니다.

HDLC와 오류 제어

HDLC 프레임 | I 프레임은 정보 전송, S 프레임은 흐름·오류 제어, U 프레임은 번호 없는 링크 관리에 사용됩니다.

ABM (비동기 균형 모드) | HDLC에서 두 단말이 대등하게 명령과 응답을 주고받는 동작 모드입니다.

ARM (비동기 응답 모드) | HDLC에서 종국이 주국의 허가를 받아 전송하는 불균형 동작 모드입니다.

패리티 검사 (Parity) | 데이터 비트의 1 개수가 짝수 또는 홀수가 되도록 검사 비트를 덧붙이는 오류 검출 방식입니다.

CRC | 생성 다항식으로 데이터를 나눈 나머지를 검사값으로 붙이는 오류 검출 방식입니다.

해밍 코드 (Hamming Code) | 검사 비트를 배치해 오류 위치를 찾아 단일 비트 오류를 정정할 수 있는 부호입니다.

FEC (전진 오류 정정) | 재전송 없이 수신 측이 추가된 오류 정정 정보를 이용해 오류를 복구하는 방식입니다.

BEC (후진 오류 정정) | 수신 측이 오류를 검출해 송신 측에 재전송을 요청하는 방식입니다.

교착 상태

상호 배제 (Mutual Exclusion) | 한 번에 하나의 프로세스만 사용할 수 있는 자원이 존재하는 조건입니다.

점유와 대기 (Hold and Wait) | 일부 자원을 점유한 채 다른 자원의 할당을 기다리는 조건입니다.

비선점 (No Preemption) | 할당된 자원을 다른 주체가 강제로 회수할 수 없는 조건입니다.

환형 대기 | 프로세스들이 원형으로 서로의 자원을 기다리는 조건입니다.

반입·배치·교체 전략

반입 (Fetch) | 보조기억장치의 자료를 주기억장치로 가져올 시점을 정합니다.

배치 (Placement) | 새로 적재할 자료를 메모리의 어느 영역에 놓을지 정합니다.

교체 (Replacement) | 공간이 부족할 때 내보낼 페이지나 자료를 정합니다.

정보시스템 구축 관리

최초·최적·최악 적합 계산

최초 적합 (First Fit) | 주소 순서대로 탐색해 처음으로 충분한 빈 영역에 배치합니다.

최적 적합 (Best Fit) | 수용 가능한 빈 영역 중 크기가 가장 작은 곳을 선택합니다.

최악 적합 (Worst Fit) | 수용 가능한 빈 영역 중 크기가 가장 큰 곳을 선택합니다.

지역성·워킹 셋·PFF

시간 지역성 (Temporal Locality) | 최근 참조한 자료나 명령을 가까운 미래에 다시 참조하는 성향입니다.

공간 지역성 (Spatial Locality) | 참조한 주소 주변의 자료나 명령을 이어서 참조하는 성향입니다.

워킹 셋 (Working Set) | 일정한 최근 구간 동안 참조한 페이지들의 집합입니다.

스래싱 (Thrashing) | 페이지 교체가 과도해 유용한 실행보다 메모리 입출력에 시간을 소비합니다.

PFF (페이지 부재 빈도 방식) | 페이지 부재율에 따라 프레임 배분을 조정합니다.

프리페이징 (Prepaging) | 앞으로 필요할 것으로 예상하는 페이지를 미리 적재합니다.

메모리 분할과 단편화

페이징 (Paging) | 논리 메모리를 일정 크기 페이지로, 물리 메모리를 같은 크기 프레임으로 나눕니다.

세그먼테이션 (Segmentation) | 코드·데이터·스택 등 논리적 의미를 가진 가변 크기 단위로 나눕니다.

내부 단편화 (Internal Fragmentation) | 이미 할당한 영역 안에서 사용하지 못하는 공간입니다.

외부 단편화 (External Fragmentation) | 빈 공간이 여러 작은 조각으로 흩어져 연속 요청을 수용하지 못합니다.

스토리지 연결 방식

DAS (직접 연결 스토리지) | 서버나 호스트에 저장장치를 직접 연결합니다.

NAS (네트워크 연결 스토리지) | 네트워크를 통해 파일 단위의 저장 서비스를 제공합니다.

SAN (스토리지 영역 네트워크) | 저장장치와 서버 사이에 블록 단위 접근을 제공하는 전용 네트워크입니다.

TCP 윈도우와 혼잡 제어

수신 윈도우 (rwnd) | 수신자가 받아들일 수 있는 데이터량을 송신자에게 알립니다.

혼잡 윈도우 (cwnd) | 송신자가 네트워크 혼잡 상태를 반영해 전송량을 제한합니다.

느린 시작 (Slow Start) | ACK가 도착하면 윈도우를 늘려 RTT 기준 대략 지수적으로 증가시킵니다.

혼잡 회피 (Congestion Avoidance) | 임계값 이후 전송량을 대략 선형적으로 늘립니다.

빠른 재전송 (Fast Retransmit) | 중복 ACK 등으로 손실을 추정해 타임아웃 전에 재전송합니다.

대칭키 알고리즘의 특징

DES (데스) | 64비트 블록과 유효 56비트 키를 사용하는 구형 대칭키 암호입니다.

AES (고급 암호 표준) | 블록은 128비트이며 키는 128·192·256비트입니다.

SEED (시드) | 기본 SEED 규격은 128비트 블록과 128비트 키를 사용합니다.

ARIA (아리아) | 128비트 블록과 128·192·256비트 키를 지원하는 국산 블록 암호입니다.

RC4 (알시포) | 키스트림과 평문을 XOR하는 구형 스트림 암호입니다.

비대칭 암호와 키 합의

RSA (알에스에이) | 소인수분해의 어려움과 관련된 공개키 암호 체계입니다.

Diffie-Hellman (디피-헬만) | 공개된 통신으로 공통 비밀값을 합의하는 키 합의 방식입니다.

ECC (타원곡선 암호) | 타원곡선상의 이산대수 문제를 활용하는 공개키 기술 계열입니다.

ElGamal (엘가말) | 이산대수 문제를 기반으로 하는 공개키 암호 체계입니다.

해시 함수의 출력 길이

MD5 (엠디파이브) | 128비트의 해시값을 출력하는 구형 해시 함수입니다.

SHA-1 (샤 원) | 160비트의 해시값을 출력하는 구형 해시 함수입니다.

SHA-256 (샤 이백오십육) | SHA-2 계열이며 256비트 해시를 출력합니다.

SHA-384 (샤 삼백팔십사) | SHA-2 계열이며 384비트 해시를 출력합니다.

SHA-512 (샤 오백십이) | SHA-2 계열이며 512비트 해시를 출력합니다.

HAS-160 (해즈 백육십) | 160비트 출력을 사용하는 국내 개발 해시 함수입니다.

정보시스템 구축 관리

벨-라파듈라·비바·클라크-윌슨

Bell-LaPadula (벨-라파듈라) | 기밀성을 위해 보안 등급 간 정보 흐름을 제한합니다.

Biba (비바) | 무결성을 위해 신뢰도가 낮은 정보가 높은 정보를 오염시키지 못하게 합니다.

Clark-Wilson (클라크-윌슨) | 검증된 거래 절차와 직무 분리로 무결성을 유지합니다.

통합 보안·무선 보안·유출 방지

WIPS (무선 침입 방지) | 비인가 무선 접속과 무선 환경의 침입을 탐지·차단합니다.

UTM (통합 위협 관리) | 방화벽·침입 방지·VPN 등 여러 보안 기능을 통합합니다.

ESM (통합 보안 관리) | 여러 보안 시스템의 상태·이벤트를 중앙에서 관리합니다.

SIEM (보안 정보·이벤트 관리) | 다양한 로그와 이벤트를 수집·상관 분석합니다.

DLP (데이터 유출 방지) | 중요 정보의 외부 반출을 탐지하거나 통제합니다.

정보시스템 구축 관리

CVE·CWE·CVSS·CC

CVE (공개 취약점 식별 체계) | 공개된 개별 취약점을 공통 식별자로 구별합니다.

CWE (공통 약점 목록) | 취약점의 원인이 되는 소프트웨어·하드웨어 약점 유형을 분류합니다.

CVSS (공통 취약점 점수 체계) | 취약점의 기술적 심각도를 일관된 척도로 평가합니다.

CC (공통 평가 기준) | 정보보호 제품의 보안 기능과 보증을 평가하는 국제 공통 기준입니다.

IPSec의 AH·ESP·IKE와 모드

AH (인증 헤더) | IP 패킷의 무결성과 데이터 출처 인증을 제공합니다.

ESP (보안 페이로드 캡슐화) | 암호화에 의한 기밀성과 설정에 따른 무결성·인증을 제공합니다.

IKE (인터넷 키 교환) | 상대를 인증하고 키와 보안 연결 설정을 협상합니다.

전송 모드 (Transport Mode) | 주로 원래 IP 패킷의 상위 계층 데이터 부분을 보호합니다.

터널 모드 (Tunnel Mode) | 원래 IP 패킷을 내부에 넣고 새 IP 패킷으로 전달합니다.

플러딩·반사·패킷 처리 공격

SYN Flooding (신 플러딩) | 다량의 연결 요청으로 TCP 연결 대기 자원을 소모시킵니다.

UDP Flooding (유디피 플러딩) | 다량의 UDP 패킷으로 회선이나 처리 자원을 소모시킵니다.

DRDoS (분산 반사 서비스 거부) | 여러 반사 서버의 응답이 피해자에게 모이도록 유도합니다.

Ping of Death (죽음의 핑) | 비정상적으로 큰 IP 데이터그램의 처리 취약점을 이용하는 고전적 공격입니다.

Teardrop (티어드롭) | 잘못 겹치는 IP 조각 등으로 재조립 취약점을 악용합니다.

느린 HTTP와 해시 자원 고갈

Slowloris (느린 HTTP 헤더) | HTTP 헤더를 끝내지 않아 서버가 연결을 기다리게 합니다.

RUDY (느린 HTTP 본문) | 큰 요청 본문을 예고하고 매우 천천히 보내 연결을 유지합니다.

Slow HTTP Read (느린 응답 읽기) | 클라이언트가 응답을 지나치게 천천히 읽어 서버 자원을 붙잡습니다.

Hash DoS (해시 서비스 거부) | 다수의 해시 충돌로 자료 처리의 계산량을 늘립니다.

표적과 유입 경로로 보는 공격

스피어 피싱 (Spear Phishing) | 특정 개인이나 조직을 겨냥해 맞춤형으로 속이는 피싱입니다.

스미싱 (Smishing) | SMS 등 문자 메시지를 이용한 피싱입니다.

큐싱 (Quishing) | QR 코드를 이용해 가짜 접속 경로로 유도하는 피싱입니다.

공급망 공격 (Supply Chain Attack) | 개발·업데이트·납품 등 신뢰하는 공급 경로를 악용합니다.

APT (지능형 지속 위협) | 특정 대상을 상대로 장기간 여러 수단을 조합하는 위협입니다.

제로데이 공격 (Zero-day Attack) | 충분한 패치·대응이 마련되지 않은 취약점을 악용합니다.

사전·대입·스터핑·레인보우 테이블

사전 공격 (Dictionary Attack)	사용할 가능성이 높은 단어·후보 비밀번호를 시도합니다.
무차별 대입 (Brute Force)	정한 탐색 공간의 가능한 조합을 폭넓게 시도합니다.
하이브리드 공격 (Hybrid Attack)	사전 후보에 숫자·기호 등의 변형을 결합합니다.
크리덴셜 스터핑 (Credential Stuffing)	다른 서비스에서 유출된 ID·비밀번호 조합을 재사용합니다.
레인보우 테이블 (Rainbow Table)	사전 계산한 해시 연결 정보를 이용해 후보 비밀번호를 찾습니다.

블루프린팅·재킹·스나핑·버깅

블루프린팅 (BluePrinting) | 블루투스 기기의 특성을 수집해 종류·모델 등을 식별합니다.

블루재킹 (BlueJacking) | 주변 블루투스 기기에 원치 않는 메시지 등을 보냅니다.

블루스나핑 (BlueSnarfing) | 블루투스 취약점을 통해 기기의 데이터를 무단으로 가져갑니다.

블루버깅 (BlueBugging) | 취약점을 악용해 기기의 기능에 무단 접근·제어합니다.

XSS의 반사·저장·DOM 유형

Reflected XSS (반사형) | 요청에 포함된 입력이 응답에 안전하지 않게 반영됩니다.

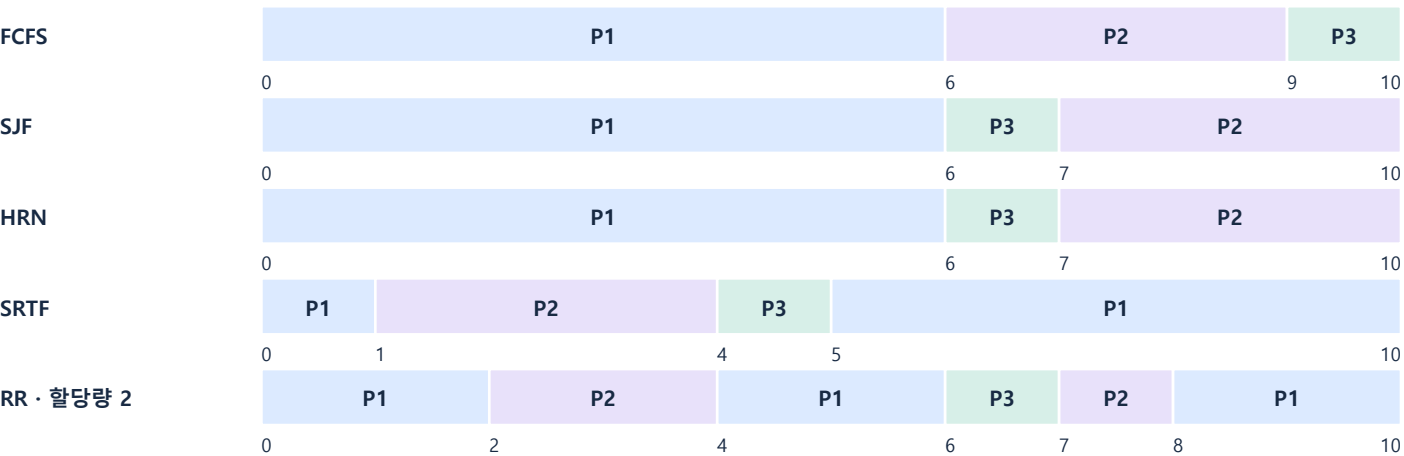
Stored XSS (저장형) | 저장된 악성 입력이 다른 사용자의 화면에서 실행됩니다.

DOM-based XSS (DOM 기반) | 브라우저 코드가 신뢰하지 않은 값을 위험한 DOM 조작에 사용합니다.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
FCFS	0 / 5 / 6	3.67	7.00
SJF	0 / 6 / 3	3.00	6.33
HRN	0 / 6 / 3	3.00	6.33
SRTF	4 / 0 / 1	1.67	5.00
RR · 할당량 2	4 / 4 / 3	3.67	7.00

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

FCFS: 도착 순서대로 실행합니다.

SJF: 시각 6에 도착해 있는 P2(3)와 P3(1) 중 짧은 P3를 먼저 선택합니다.

HRN: 시각 6의 응답률: P2=(5+3)/3=8/3, P3=(3+1)/1=4 → P3 선택.

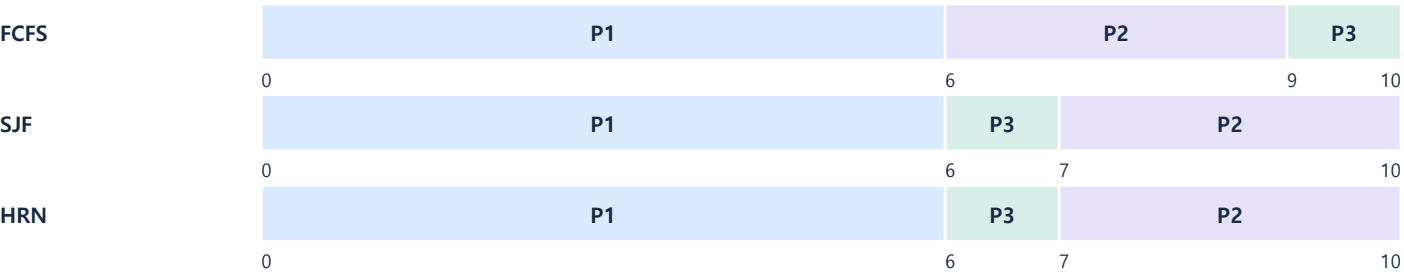
SRTF: 시각 1에 P2가 P1을 선점합니다. 시각 3에는 P2·P3의 잔여시간이 같아 현재 작업 P2를 유지한다고 가정합니다.

RR · 할당량 2: 실행 중 도착한 작업은 준비 큐에 넣고, 할당량 후 미완료 작업은 큐 맨 뒤로 보냅니다.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
FCFS	0 / 5 / 6	3.67	7.00
SJF	0 / 6 / 3	3.00	6.33
HRN	0 / 6 / 3	3.00	6.33

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

FCFS: 도착 순서대로 실행합니다.

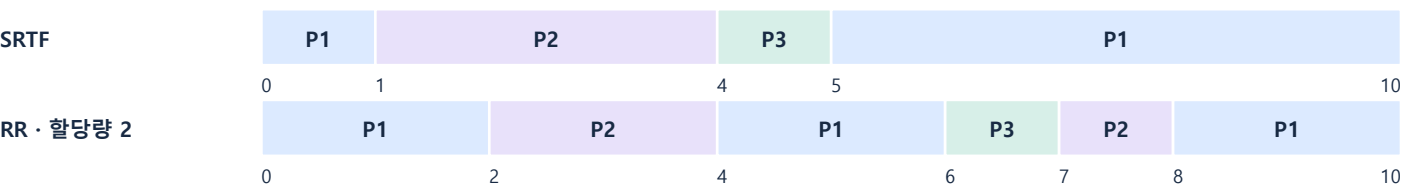
SJF: 시각 6에 도착해 있는 P2(3)와 P3(1) 중 짧은 P3를 먼저 선택합니다.

HRN: 시각 6의 응답률: P2=(5+3)/3=8/3, P3=(3+1)/1=4 → P3 선택.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
SRTF	4 / 0 / 1	1.67	5.00
RR · 할당량 2	4 / 4 / 3	3.67	7.00

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

SRTF: 시각 1에 P2가 P1을 선점합니다. 시각 3에는 P2·P3의 잔여시간이 같아 현재 작업 P2를 유지한다고 가정합니다.

RR · 할당량 2: 실행 중 도착한 작업은 준비 큐에 넣고, 할당량 후 미완료 작업은 큐 맨 뒤로 보냅니다.

계산 예제 - 네트워크·브로드캐스트 주소

호스트 비트 h = 32 - 프리픽스 | 전체 주소 = 2^h | 일반적인 호스트 수 = 2^h - 2

블록 크기 = 256 - 변화 옥텟의 마스크. IP가 들어가는 블록의 시작은 네트워크, 끝은 브로드캐스트입니다.

입력 / 마스크	네트워크	브로드캐스트	사용 가능한 주소
192.168.10.77/26 **255.255.255.192**	192.168.10.64	192.168.10.127	192.168.10.65 ~ 192.168.10.126
172.16.9.130/22 **255.255.252.0**	172.16.8.0	172.16.11.255	172.16.8.1 ~ 172.16.11.254

77/26: 블록 64 → 64~127 → 네트워크 .64, 브로드캐스트 .127. 9/22: 세 번째 옥텟 블록 4 → 8~11 → 네트워크 172.16.8.0, 브로드캐스트 172.16.11.255.

주소 검산: 네트워크는 IP AND 마스크. 브로드캐스트는 호스트 비트를 모두 1로 합니다. /31 점대점 링크에는 일반적인 2^h-2 공식을 적용하지 않습니다.

계산 예제 - 서브넷 분할

같은 크기 2^n개로 분할: 기존 프리픽스에 n을 더합니다. /27을 2분할하면 /28, 블록 16주소, 일반 호스트 14개입니다.

10.20.4.64/27을 2분할	브로드캐스트	사용 가능 범위
10.20.4.64/28	10.20.4.79	10.20.4.65 ~ 10.20.4.78
10.20.4.80/28	10.20.4.95	10.20.4.81 ~ 10.20.4.94

10.20.4.82는 둘째 블록 80~95에 포함됩니다. 정답: /28, 브로드캐스트 10.20.4.95.

계산 예제 - 페이지 교체

빈 프레임 3개, 참조 순서 1 2 3 1 4 2 5

방식 / 참조	1	2	3	1	4	2	5
FIFO 프레임	1/-/-	1/2/-	1/2/3	1/2/3	4/2/3	4/2/3	4/5/3
FIFO 부재	O	O	O	-	O	-	O
LRU 프레임	1/-/-	1/2/-	1/2/3	1/2/3	1/4/3	1/4/2	5/4/2
LRU 부재	O	O	O	-	O	O	O

초기 적재도 부재에 포함: FIFO 5회, LRU 6회. FIFO는 적재 순서, LRU는 최근 사용 시점을 기준으로 교체합니다.